Sameer AI Platform — Microservices Architecture

Goal

Build a scalable AI platform with:

- WhatsApp AI Bot
- Spring AI Gateway
- Ollama LLM
- Python AI services
- Future memory/RAG/vector DB
- VPS deployment

Final Architecture

```
WhatsApp
   ↓
Node.js WhatsApp Service (Baileys)
   ↓
Spring AI Gateway Service
   ↓
├── Ollama (qwen2.5:1.5b)
├── Python AI Service (FastAPI)
├── MongoDB
├── Vector DB
└── Future AI Agents
```

Project Structure

```
sameer-ai-platform/
│
├── services/
│   ├── whatsapp-service/
│   ├── ai-gateway/
│   ├── python-ai-service/
│   └── memory-service/
│
├── docker/
```

```
├── scripts/
└── docs/
```

STEP 1 — Create Root Folder

```
mkdir -p ~/Desktop/sameer-ai-platform/services
cd ~/Desktop/sameer-ai-platform
```

STEP 2 — Create WhatsApp Service

```
cd services
mkdir whatsapp-service
cd whatsapp-service
```

Move your existing bot files here.

Structure:

```
whatsapp-service/
├── bot.js
├── package.json
├── auth/
└── node_modules/
```

STEP 3 — Create Spring AI Gateway

Generate Spring Boot Project

Open:

https://start.spring.io

Use:

Setting	Value
Project	Maven
Language	Java
Java	21
Group	com.sameer
Artifact	ai-gateway

Dependencies:

- Spring Web
- Lombok
- Spring AI Ollama

Download project.

STEP 4 — Extract Project

```
cd ~/Desktop/sameer-ai-platform/services
unzip ~/Downloads/ai-gateway.zip
```

STEP 5 — Install Ollama Model

```
ollama pull qwen2.5:1.5b
```

Test:

```
ollama run qwen2.5:1.5b
```

STEP 6 — Configure Spring AI

application.yml

Location:

```
services/ai-gateway/src/main/resources/application.yml
```

Content:

```
spring:
  application:
    name: ai-gateway

  ai:
    ollama:
      base-url: http://localhost:11434
      chat:
        model: qwen2.5:1.5b

server:
  port: 8080
```

STEP 7 — Create AI Controller

Create:

```
services/ai-gateway/src/main/java/com/sameer/aigateway/controller/
ChatController.java
```

Code:

```
package com.sameer.aigateway.controller;

import org.springframework.ai.chat.client.ChatClient;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/ai")
public class ChatController {

    private final ChatClient chatClient;

    public ChatController(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }
```

```
    @GetMapping("/chat")
    public String chat(@RequestParam String message) {

        return chatClient.prompt()
                .user(message)
                .call()
                .content();
    }
}
```

STEP 8 — Run Spring AI Service

```
cd ~/Desktop/sameer-ai-platform/services/ai-gateway
mvn spring-boot:run
```

Test API:

```
curl "http://localhost:8080/api/ai/chat?message=hello"
```

AI should reply.

STEP 9 — Connect WhatsApp Bot To Spring AI

Inside:

```
services/whatsapp-service/bot.js
```

Install axios:

```
npm install axios
```

Use:

```
const axios = require("axios");

async function askAI(message) {
```

```
    try {

        const response = await axios.get(
            "http://localhost:8080/api/ai/chat",
            {
                params: {
                    message
                }
            }
        );

        return response.data;

    } catch (err) {

        console.log(err.message);

        return "AI service offline";
    }
}
```

STEP 10 — Future Python AI Service

Later:

```
cd ~/Desktop/sameer-ai-platform/services
mkdir python-ai-service
```

Tech stack:

- FastAPI
- LangChain
- Transformers
- Whisper
- Image AI

Future Services

Memory Service

Use:

- MongoDB
- Redis

Vector Database

Use:

- Qdrant
- ChromaDB

Message Queue

Later add:

- Kafka
- RabbitMQ

VPS Deployment Later

Deploy:

```
Ubuntu VPS
├── Docker
├── Spring AI
├── Ollama
├── WhatsApp Service
└── MongoDB
```

Recommended Learning Order

Phase 1

- WhatsApp service
- Spring AI gateway
- Ollama integration

Phase 2

- Docker
- VPS deployment
- MongoDB memory

Phase 3

- Python AI service
- RAG
- Vector DB
- Voice AI

Phase 4

- AI agents
- Cybersecurity assistant
- Multi-model orchestration

Current Goal

Complete:

```
WhatsApp → Spring AI → Ollama
```

before adding more services.